

Hearthvale

A cozy voxel town-builder for AYN Thor Max

WORKING TITLE / DRAFT 0.1 / 6 SEPTEMBER 2026



SUPPLIED VISUAL REFERENCE — NOT GAME FOOTAGE

A small valley to shape.
A village to make your own.
Little lives unfolding inside it.

Personal, controller-first and offline. An agent-led implementation plan with explicit prototype gates. Hardware claims are sourced; performance budgets are targets, not test results.

Inside the valley

The creative brief comes first. Device constraints, implementation details, and the agent handoff follow in the second half. This draft preserves the requested scope while separating it from the first playable build.

- 1. Project brief 3
- 2. Play experience 4
- 3. Visual and audio direction 4
- 4. The valley, terrain, and water 6
- 5. Buildings that remain yours 7
- 6. Humans, animals, and places 8
- 7. Controller-first interaction 9
- 8. Thor Max hardware and deployment target 11
- 9. Engine and plugin decision 12
- 10. Runtime architecture 13
- 11. Performance and visual budgets 14
- 12. Saves, undo, and resilience 15
- 13. Agent-assisted development and MCP 16
- 14. Milestones and acceptance gates 17
- 15. Risks, decisions, and first handoff 19
- Sources and research notes 20
- Visual reference appendix 21

Reading guide: sections 1–7 describe the game; sections 8–13 define the technical plan; sections 14–15 are the implementation roadmap. All numerical budgets are proposed starting points.

1. Project brief

The promise

Create a detailed, inviting village inside a small, fully sculptable voxel valley. Draw and stretch buildings, let the game supply attractive architectural details, then change any of those details yourself. Add a café, an inn, a playground, or a quiet garden and watch people and animals use the place.

The central reward is authorship: “I made this corner of the world, and I like spending time here.” There is no requirement to optimise a settlement or finish a checklist.

Requirements established with the player

Area	Design requirement
Building	Stretchable procedural buildings, with individually editable automatic details.
Landscape	A small valley; sculpting, material painting, forests, rivers, caves, and overhangs.
Life	Humans and animals; places with recognisable uses, including cafés, playgrounds, and inns.
Controls	Single-screen, controller-only gameplay, including when the Thor is connected to a TV.
Appearance	Detailed voxel forms, attractive lighting, and multiple architectural styles; nothing properly modern.
Development	Personal project; an agent does most implementation, with the player directing and reviewing.

Proposed defaults, not additional locked requirements

Unlimited materials; no money, upkeep, failure, hunger, death, mandatory quests, or real-world waiting. Everything available without progression gates. The initial experience is exterior-focused, with decorative window interiors rather than furnished enterable buildings. Water is an editable scenic system, not a general fluid simulation. These defaults keep the full creative brief while avoiding several unrelated simulation projects.

Single-player and offline at runtime. AI assists development; it is not required to generate buildings or operate villagers during play. A desktop build is a development convenience, not the target whose performance determines success.

Scope boundary

The vision includes all the requested terrain freedoms and inhabited places. They arrive in stages, not in one first implementation. Infinite worlds, multiplayer, structural collapse, traffic simulation, full interiors, and general-purpose fluid physics are outside this draft. A walk-through camera is a possible later addition, not a prerequisite for building.

Success test: a player can sit down with only a controller, make a pleasing change within a minute, undo it without anxiety, and return later to exactly the village they left.

2. Play experience

The core loop

Shape → build → personalise → bring to life → enjoy.

Start with an attractive generated valley or a simpler blank landscape. Sculpt a terrace, draw a cottage, stretch its roof, move a window, paint a path, and add somewhere to sit. Assign a place a purpose and watch a small activity appear. Pull the camera back, adjust the light, and decide whether to build another corner or just watch.

Short sessions should be complete in themselves. Moving a chimney, planting a little grove, or making a bench overlook a river is a legitimate session, not preparation for the real game.

A representative first session

The player opens a riverside clearing in late-afternoon light. A small starter cottage demonstrates the editing handles. They widen it, replace two windows, and move the front door toward the river. A path drawn from the door curves toward a new terrace. They assign the building “Café” and place two outdoor tables. Visitors approach, sit, and chat. A duck wanders along the bank. The player carves a sheltered recess into the opposite cliff and places a lantern there.

The game has shown its entire identity: landscape freedom, forgiving architecture, personal detail, and life without management demands.

Design pillars

Generous tools. Large gestures produce something attractive quickly. Fine controls exist without being required. Sensible snapping, visible previews, and predictable undo matter more than a huge catalogue.

Personal edits win. Automatic generation proposes; it does not silently replace the player's choices. Resizing, recolouring, duplicating, saving, and loading must respect that rule.

Life follows design. A place's purpose changes what happens there. It does not create a bill, an unmet need, or a permanent warning badge.

Small world, convincing depth. A finite valley can contain layered ridges, woodland, a stream, a settlement, and a few discoveries. Distant scenery supplies atmosphere, not a promise of an entire continent to edit.

Comfort features

A persistent undo/redo route, local autosaves, optional activity pause, adjustable camera sensitivity, remappable actions, readable interface scaling, separate audio sliders, and optional haptics. No compulsory camera shake, motion blur, flashing alerts, or building timers. A hide-interface view and a few saved camera bookmarks support simply enjoying the result.

3. Visual and audio direction

What to take from the references

The supplied images suggest warm terraced stone, dark pine silhouettes, turquoise water, large arches, dramatic valley walls, and atmospheric separation between foreground and distant ridges. Borrow those relationships rather than the enormous visible world or the exact assets.

The desired look is detailed and tactile, not a field of oversized toy cubes. Use a hierarchy: clear large landforms first, readable buildings second, small architectural and material variation third. Voxel steps should describe the form instead of becoming uniform visual noise.

Reference status: the supplied images are inspiration, not screenshots of Hearthvale. Visible labels include AURELION, Kingsfall Basin, and The Old Crossing; the first image includes a Reddit attribution to u/LostRequirement4828 in r/codex. Do not redistribute their scenery as project assets or imply it is our work.

Architectural families

Family	Visual vocabulary	Delivery
Riverside cottage	Rough stone, plaster, timber, steep roofs, shutters, flower boxes, warm windows.	First complete kit.
Alpine / northern village	Deep eaves, timber balconies, stone foundations, carved wood, restrained colour.	Second kit; validates the style system.
Old-town fantasy	Half-timber façades, irregular rooflines, towers, archways, hanging signs, narrow streets.	Later content expansion.

Treat styles as interchangeable data: materials, roof profiles, detail collections, placement rules, and variation ranges. A café is a function, not an architectural style. The same purpose must work in more than one style. Individual buildings can override the valley's default style without changing neighbours.

Lighting strategy

Begin with one shadow-casting sun, a sky-and-ambient lighting setup, local geometric/vertex shading, a modest fog gradient, and a small number of nearby lamps. Cave shading needs explicit treatment: distinguish enclosed surfaces from outdoor ones rather than allowing uniform sky ambience to illuminate every tunnel. Prototype this alongside terrain editing.

Godot's Mobile renderer supports the basic lighting approach, reflection probes, glow, and ordinary fog, but not VoxelGI, SDFGI, screen-space reflections, or volumetric fog. Do not make those effects part of the required visual target. Baked whole-valley lighting is also not the proposed solution for geometry the player continuously changes. [S4]

Create attractive daytime and golden-hour presets first. A manually adjustable time-of-day slider can follow; a slow day/night cycle is optional. Water should prioritise colour, shoreline definition, animated surface detail, and a believable sky reflection over perfect reflections of every building.

Art production boundary

Build the first kit from a small number of reusable, consistently scaled parts. Use procedural geometry for shells and roofs, and authored or generated mesh pieces for windows, doors, signs, plants, and furniture. Keep style manifests and attachment dimensions explicit so new art can replace placeholders without rewriting building logic.

The agent can implement generators, assemble kits, and validate imports; the player approves silhouettes, materials, lighting, and animations. Start inhabitants with a minimal reusable rig and a small activity set. Record the origin and permitted use of every external mesh, texture, font, and audio file. Supplied inspiration images are reference material, never an asset library to extract from.

Sound

A quiet layered ambience: wind, birds, water, distant conversation, occasional animals, and softened building feedback. Give rivers, woods, and inhabited squares different acoustic identities. Keep music optional and sparse. The village must remain pleasant with music muted or external music playing.

4. The valley, terrain, and water

World size and representation

Proposed first playable: **128 × 128 metres**, with a riverbank and enough relief to test a tunnel and a bridge. Proposed personal-game target: **256 × 256 metres**, with roughly **96 metres of editable vertical range**, divided between below-ground depth and peaks. These are starting budgets, not measured hardware limits.

Use an explicit editable valley boundary. A lightweight mountain backdrop outside it may extend the view, but is scenery, not secretly inaccessible buildable land. Larger maps require a later benchmark rather than a default promise.

The terrain must be genuinely volumetric: caves and overhangs cannot be reduced to a single ground-height value. Voxel Tools explicitly supports editable volumetric terrain and converts voxel data into chunk meshes. [S5]

Terrain tools

Raise/add, lower/remove, smooth, flatten to a sampled level, terrace, and paint surface materials. Add an accessible tunnel/arch brush for carving through a cliff, plus vegetation and rock brushes with density, scale, and variation controls. A broad brush should create a hillside; a smaller one should clean up its silhouette.

Expose brush radius and strength with controller-friendly steps and a clear footprint. Sampling a surface provides a flatten height or material without typing. Terrain strokes are transactions: a full stroke is one undo action, not hundreds of controller taps.

Trees, buildings, paths, water, and ground are separate edit layers. Terrain brushes do not accidentally delete buildings. Where excavation removes a building's support, preserve the building and preview foundation adjustment; do not introduce collapse physics. Where an edit intersects existing objects, show the effect before committing and offer an explicit affected-object action.

Water: editable scenery with defined rules

Proposed compromise: let the player create and reshape lakes, streams, riverbeds, and waterfalls without solving general fluid dynamics. A stream is an editable centreline with width, bed depth, surface height, and a visual flow direction. A lake is a bounded water region with a chosen level. A waterfall joins authored upper and lower water regions.

The river tool can preview a carved bed and banks; accepting it commits terrain and water together as one undoable operation. Later terrain edits recompute only the affected shore or channel section. Water visuals must be clipped against the actual terrain and water-region bounds, not drawn as a giant plane through caves.

A carved breach does not automatically flood the valley. The editor previews an uncontained region and lets the player reduce its level, alter its boundary, or deliberately extend it. Different water heights remain separate authored bodies until the player connects them. Test a cave beneath a lake and a tunnel beside a stream before declaring the water system complete.

This still supports all requested landscape tools; it does not promise physically correct damming, underground flooding, erosion, or pressure. Voxel Tools' fluid models do not implement fluid behaviour on their own. [S6]

Generation

Start with a small set of art-directed valley templates plus a seed: river bend, wooded basin, terraced gorge. Vary hills, rock layers, tree groups, and clearings, while keeping a useful starting build area. Keep the generator version with each save; a future algorithm update must not rewrite an existing valley.

5. Buildings that remain yours

Primary interaction

Choose a building tool, place an initial footprint, and stretch width, depth, and height using large visible handles. The building receives a suitable roof, wall treatment, windows, doors, and restrained decoration. Select any generated detail to move, replace, recolour, duplicate, suppress, or lock it.

Initial structural forms are rectangular volumes with gabled roofs. Add connected wings, more roof profiles, round towers, and arches after the first generator is dependable. Start with explicit roof-joining rules rather than promising arbitrary flawless boolean unions.

The non-destructive contract

A building is a persistent design recipe, not a discarded mesh-generation input. It contains stable identifiers for its volumes and surfaces, dimensions, transform, style, seed, roof parameters, purpose, and a separate collection of manual overrides. Meshes and automatically placed details are derived outputs.

Detail state	Behaviour during regeneration
Automatic	May be regenerated according to the building's current rules.
Modified / locked	Preserve the chosen asset and settings; resolve its attachment against the edited surface.
Suppressed	A persistent exclusion prevents the generator from replacing a detail the player removed.
Manually added	Remains a separately identified attachment; never becomes disposable generated decoration.

Use surface-local anchors: a stable wall or roof ID, a local position, and an attachment policy. A door may keep a measured offset from a wall corner; a dormer may track a proportion of the roof width. Do not use “window number 3” as the permanent identity when the number of windows can change.

Conflict policy: when shrinking or deleting a surface makes an override invalid, keep the override in the building data and show it in a small “needs placement” tray. Offer reposition, reattach, or remove. Never silently delete it, move it onto an unrelated wall, or erase it because a different roof was selected. The entire action remains undoable.

Resizing example and acceptance test

Create a cottage with six automatic windows. Move one, replace one, remove one, and add a flower box. Stretch the cottage wider, then shorter; switch materials; save and reload. Valid edits persist, the deleted window does not return, and genuinely orphaned attachments are recoverable. Undo restores the previous recipe and overrides exactly.

This test is a milestone gate, not polish to add after the generator.

Relationship to voxels

Use a coarser world grid for terrain and a finer building-local construction grid for stepped walls, roofs, stonework, and trim. A first test might use 0.5-metre terrain cells and 0.125- to 0.25-metre building units. Decorative meshes can be finer without increasing the resolution of the whole valley.

Render building forms as batched, exposed-surface geometry, with selective mesh details where appropriate. Everything need not share one global cube grid to look coherent. Full manual voxel surgery on building shells is deferred; the required freedom is individual control over generated architectural details and structural handles.

Foundations and connections

Generate foundations from local ground samples without secretly rewriting the terrain. Offer stone plinths, short steps, or supports where appropriate. Paths may connect to doors, and bridge endpoints may snap to banks or terraces. Manual placement always remains available; an automatic suggestion must not turn into a restrictive zoning rule.

6. Humans, animals, and places

An inhabited scene, not an economy

Humans and animals provide readable, pleasant activity. Their behaviour should make the player's choices visible: an outdoor café produces seated conversation, an inn produces arrivals and departures, and a playground produces play. There are no workforce quotas, happiness meters, or consequences for leaving the game alone.

The player assigns a purpose to a building or drawn outdoor area. Useful props then offer activity locations. Suggested props can be added automatically, but remain editable under the same personalisation rules.

Place	Suggested activity	Minimum functional anchors
Home	Leave, return, pause at the doorway or garden.	Reachable entrance.
Café / tea garden	Approach a table, sit, drink, converse.	Entrance or area access plus a chair/table slot.
Inn	Arrive, linger by a sign or bench, enter or leave.	Entrance plus waiting point.
Playground	Use a swing, play nearby, pause with companions.	Safe approach and animation-aligned play slots.
Garden / park	Sit, stroll, look at scenery.	Reachable paths or sitting points.

Place	Suggested activity	Minimum functional anchors
Animal area / pond	Graze, perch, wander, swim within a bounded area.	Habitat region and species-appropriate activity points.

Behaviour model

Use a small state machine: idle, choose activity, travel, perform activity, and depart/rest. An activity point describes who can use it, approach position, orientation, duration, capacity, and animation. Reservations prevent two people occupying the same seat. Character variation can come from appearance, preferred activities, and small timing differences rather than complex personal simulation.

Prototype with four humans and two ducks. Grow toward roughly 24 visible humans and 12 animals in the normal benchmark scene, with a density control. These are adjustable targets, not a minimum population every world must have.

Begin with one human rig and one animal family. Cats, dogs, sheep, and birds can be added as separate content milestones. Avoid committing to a dozen rigs and animation sets before the building tools are enjoyable.

Navigation through an editable world

Rebuild navigation in affected regions after an edit settles, not on every brush sample. Use simplified walkable surfaces rather than tiny decorative geometry. Bridge and cave layers need genuine 3D connectivity; a single height map is not sufficient. Godot supports navigation-mesh baking and region-based navigation, but the project's dirty-region scheduling and source-geometry extraction are custom work. [S12]

When a path, seat, or bridge is removed, cancel the associated reservation and activity. The character waits, reroutes, or relocates safely out of view. No falling to death, permanent pathfinding loops, or punitive accessibility warnings. A controller-accessible debug overlay can reveal unreachable anchors during development.

7. Controller-first interaction

One interface for handheld and TV

All normal actions, settings, saves, loading, and editing must be possible without touching either display or using a mouse. Build a world-space cursor and contextual selection system, not an operating-system mouse emulator. Tool focus, highlighted targets, and the next action must always be visible.

Use action names internally and remappable bindings. The examples below use Xbox-style labels only as a starting layout; displayed glyphs must follow the selected controller layout. The Thor's own buttons and an external pad must both be tested.

Input	World / editing behaviour
Left stick	Move the world cursor with gentle camera follow; while a handle is grabbed, adjust its constrained parameter.
Right stick	Orbit and tilt around the current focus.

Input	World / editing behaviour
LT / RT	Zoom out / in, including while inspecting a building.
A	Select, place an anchor, grab a handle, or commit the current preview.
B	Cancel the current preview first; otherwise back out one level.
X	Open object details; inside handle editing, cycle the available handles.
Y	Open/close the main tool palette.
D-pad	Contextual stepped values: brush size, variation, or rotation; always labelled on screen.
LB / RB	Undo / redo in the world. In menus, switch labelled tabs instead.
L3 / R3	Optional precision toggle / focus selected object; provide menu equivalents.
Start	Pause and settings; safe cancel or commit handling for an active edit.

Menus suspend world editing. The same press must never activate a menu and modify terrain behind it. Avoid mandatory long holds and complicated modifier chords; provide hold/toggle choices where a brush benefits from continuous application.

Making precision practical

Magnetise selection to relevant walls, roofs, handles, and nearby details. Provide an explicit overlapping-target cycle. An attachment moves on its selected surface plane, not freely through three dimensions. Resizing uses a constrained axis with visible extent and snap increments. A precision mode reduces movement speed and step size.

Cave editing includes an adjustable cutaway view or slice plane so the cursor can reach interior surfaces without blindly excavating the roof. The same cutaway approach can help edit details obscured by foreground scenery. The render view, terrain hit-testing, and hidden-surface selection must agree.

Camera and interface

Default to a perspective orbit camera with bounded pitch, smooth but responsive movement, and a reset-to-comfortable-view action. Keep narrow-field-of-view and near-orthographic presentation as visual options to evaluate, not assumptions.

Proposed minimum common text size: about 24 pixels at a 1080p interface canvas, with larger presets and a couch-safe margin setting. Judge actual readability on the 6-inch screen and the TV. Naming is optional: generate sensible names, and provide an in-game controller keyboard only where custom text is useful.

Acceptance route: cold launch → load valley → carve tunnel → stretch cottage → move window → assign café → undo/redo → save → quit, without touch input.

8. Thor Max hardware and deployment target

Verified hardware baseline

The table combines AYN's original specification graphic with its current product listing. The graphic is reproduced by Retro Dodo and was inspected directly. Storage is specifically batch-dependent. [S1, S2]

Component	Thor Max specification
SoC	Qualcomm Snapdragon 8 Gen 2, 4 nm.
CPU	Eight cores; AYN lists 1 at 3.2 GHz, 4 at 2.8 GHz, and 3 at 2.0 GHz.
GPU	Adreno 740; AYN lists 680 MHz.
Memory	16 GB LPDDR5X; shared system/graphics memory, not 16 GB of dedicated desktop VRAM.
Main display	6-inch AMOLED touchscreen, 1920 × 1080 in landscape, up to 120 Hz.
Secondary display	3.92-inch AMOLED touchscreen, 1240 × 1080 in landscape, 60 Hz. Not used by this game.
Storage	Current Max options: 512 GB or 1 TB, listed as UFS 3.1. Original Max marketing listed 1 TB UFS 4.0. Verify the actual unit.
OS / cooling	Android 13 as advertised; active cooling. Record actual firmware during testing.
Battery / charging	6,000 mAh; 27 W charging as advertised.
Connections	USB 3.1 Type-C, microSD slot, 3.5 mm audio, Wi-Fi 7, Bluetooth 5.3.
Video output	DisplayPort Alt Mode, advertised up to 4K at 60 Hz.
Controls / body	Hall-effect sticks; approximately 150 × 94 × 25.6 mm and 380 g.

Clock specifications do not guarantee sustained performance. Treat available memory, render resolution, temperature, and the cost of editing as measured constraints. A 4K display-output capability is not a 4K game-rendering target.

Deployment choices

Ship a native **Android ARM64 APK** for local installation. Use one landscape gameplay surface with no secondary-screen dependency. The initial TV workflow is the Thor's normal external-display/mirroring path; do not build a custom second Android display implementation before verifying that path on the actual firmware and dock.

Godot provides Android exports and a documented SDK/JDK setup. Pin export templates to the chosen engine version and keep signing material outside version control. Use a stable application ID and signing identity so updates can retain local data. [S7]

Test both the built-in controller and an external controller, suspend/resume, TV connection changes, audio routing, and return from Android settings. Disconnecting a controller cancels an uncommitted drag and pauses

input safely. Battery-saving mode may reduce frame rate and scenery density; it must not alter saved objects or erase detail.

9. Engine and plugin decision

Recommendation

Start with **Godot 4.7.2, typed GDScript, and the Mobile renderer**, subject to the platform spike. The official release archive lists 4.7.2 as stable and 4.8 as development builds on the research date. Do not select a development engine merely for a newer feature. [S3]

The main reason to choose Godot here is the combination of the player's preference, native Android export, a usable voxel extension route, and an editor that can be driven by a coding agent. This is a project-fit judgement, not a claim that Godot will automatically run this game faster than another engine. [S5, S7, S8]

Unity with its mobile rendering pipeline remains a reasonable alternative if a concrete blocker appears; its Android support is documented. However, changing engines does not supply the custom smart-building and override system. Unreal is not the preferred starting point for this narrowly scoped personal build. No comparative prototype has been run, so there is no measured engine winner. [S15]

Dependency shortlist

Tool	Role	Decision / caution
Zylann Voxel Tools	Volumetric terrain, chunk meshing, terrain edits and persistence primitives.	First dependency to prove on Thor. Prefer GDExtension if the tested package is reliable.
Godot AI — hi-godot	Community MCP bridge for live editor interaction.	Development-only. Use a tagged release and matching server/client configuration.
GUT — bitwes	Automated GDScript tests.	The project's table maps GUT 9.7.1 to Godot 4.7.x. Keep tests independent of the MCP connection.
Input Helper — nathanhead	Device detection, action rebinding helpers, controller handling.	Optional convenience. Test actual Thor and external-pad mappings; it does not create the game's UI navigation.

Voxel Tools documents Android ARM64 libraries for its GDExtension edition and use with ordinary export templates, but also calls that edition less tested than its engine-module version. Its v1.7 module release is based on Godot 4.7.2. A module release is not proof that a particular extension download contains the correct Android debug and release binaries. Check the package, architecture, and engine pairing explicitly. [S9, S10]

GUT and Input Helper are independently documented by their maintainers. Pin exact releases after the first compatibility test; avoid accumulating extra dependencies before a problem requires them. [S11, S13]

Prefer built-ins where they are sufficient

Use Godot resources for styles and definitions, procedural meshes for building outputs, MultiMesh for repeated scenery, and Godot's navigation tools for agents. MultiMesh is suited to repeated mesh instances; group it spatially rather than placing the whole valley's vegetation in one giant visibility group. [S12, S14]

Do not adopt Terrain3D or a heightmap plugin as the sole editable-terrain system. Terrain3D is heightmap-based; holes plus separately modelled caves are not equivalent to the player's requested unrestricted volumetric sculpting. VoxelGI is a lighting feature, not a voxel-world plugin. [S4, S16]

10. Runtime architecture

Separate design data from generated output

Controller actions → preview → validated edit command → authoritative world data → local derived updates.

The world data owns terrain changes, building recipes, detail overrides, paths, water definitions, place functions, inhabitants, and presentation settings. Meshes, collision, navigation, and decoration are replaceable outputs. Saving should not depend on serialising the current scene tree as the complete world.

Component	Responsibility
WorldModel / WorldStore	Stable IDs, authoritative entities, schema and generator versions.
ToolController / Selection	Contexts, world cursor, snapping, surface targeting, preview lifecycle.
CommandHistory	Transactional apply, undo, redo, change regions and memory budget.
TerrainBackend	Wrap the selected voxel plugin; expose edit, query, snapshot and dirty-region operations.
BuildingGenerator	Recipe → shell/roof geometry and automatic attachment proposals.
OverrideResolver	Preserve edits, exclusions, anchor policies and recoverable conflicts.
PlaceSystem / ActivitySystem	Functional areas, usable anchors, reservations and ambient routines.
RebuildScheduler	Bounded mesh, collision, shoreline and navigation update jobs.
SaveService	Consistent checkpoints, recovery, migrations and backup management.

These are responsibilities, not instructions to create nine global singletons. Keep services local to a world where possible and use a small number of explicit coordinating nodes.

Chunking and detail

Start by testing 16- and 32-cell chunk edges. Only render exposed surfaces. Never create a node, rigid body, or draw call per voxel. Reuse materials and atlases. Keep terrain resolution separate from decorative building resolution.

Voxel Tools' VoxelMesherBlocky culls neighbouring hidden faces but does not automatically perform greedy meshing. Do not claim a performance budget based on optimisation the chosen mesher does not provide. Its smooth-LOD terrain path is a different approach, not a drop-in blocky-terrain LOD switch. Start with the blocky visual target and a bounded scene, then measure whether a far-view proxy is needed. [S6, S17]

Keep expensive voxel work in the native terrain backend. Use GDScript to orchestrate bounded edits and high-level generation; only move additional code into C++ after profiling identifies a real hotspot. Avoid an engine fork as the default personal-project dependency.

Asynchronous work and correctness

An edit increments affected region/entity revisions. Jobs read immutable input snapshots and return results tagged with that revision. Apply results on the appropriate main-thread boundary only if the revision still matches; discard stale jobs after later edits or undo.

Show a lightweight preview immediately. Coalesce a continuous brush stroke and rebuild affected geometry at a bounded rate, with a final rebuild on commit. Do not enqueue one whole-world rebuild per input sample. Collision and navigation may settle afterward, but the player must see when an operation is pending and must not interact with mismatched old/new geometry.

Stable data sketch

A BuildingRecord needs: id, schema_version, generator_version, style_id, seed, transform, volumes with stable surface IDs, roof settings, material overrides, detail overrides, suppression regions, purpose, and activity anchors. A terrain save needs base-generation identity plus absolute modified chunk data, not only a replay of every brush movement since the world began.

Use separate random streams for terrain, each building, and ambience. Adding a bird or changing a roof must not consume randomness that reshuffles unrelated objects.

11. Performance and visual budgets

Targets to validate, not promises

Aim for a stable **60 fps** normal mode, with a **30 fps** quiet/battery mode. A dependable 30 fps vertical slice is more useful than an intermittently fast scene with input stalls. Do not advertise 60 fps as achieved until the full benchmark route passes on the Thor after warm-up.

Area	Initial target / evaluation rule
Internal 3D resolution	Start at 1280 × 720; assess 1600 × 900 afterward. Keep UI at output resolution.
TV output	Begin testing at 1080p/60 output. Do not automatically render 3D at a TV's 4K resolution.
Normal scene	32 buildings, 2,000 vegetation/prop instances, 24 humans and 12 animals across the loaded valley.
Stress scene	64 buildings, 5,000 vegetation/prop instances, 32 humans and 16 animals; overview plus dense close-up views.
Lighting	One shadowed sun; initially at most four nearby non-shadowed decorative lights.
Memory	Investigate above 2.5 GiB steady process memory or 3.5 GiB transient peak; include native/GPU allocations where measurable.

Area	Initial target / evaluation rule
Editing feedback	Immediate cursor/preview feedback; visible ordinary edits ideally within 100 ms. Bound large operations.
Save behaviour	No long main-thread freeze; checkpoint progress should not block camera movement.
Frame pacing	At 60 fps, aim for 95th-percentile frame time ≤ 20 ms and 99th ≤ 33.3 ms; record misses and edit spikes separately.

The population and object counts describe fixtures, not player-facing hard caps. Prototype-derived limits may differ. Record actual visible instances, triangles, draw calls, mesh-update time, and memory alongside total object counts.

Test procedure

Use a fixed seed, fixed saved world, repeatable camera route, and repeatable edit sequence. Warm the device for 20 minutes at a recorded brightness, fan/performance mode, power state, firmware, and resolution. Then capture at least five minutes including overview, forest close-up, water, a cave, resize operations, terrain strokes, undo, and save.

Repeat in handheld and TV modes. Compare Mobile and Compatibility in the platform spike using equivalent settings. Headless tests cannot establish rendering quality or Android GPU performance. Development-PC screenshots are visual evidence only; they are not Thor benchmarks.

Optimisation order

Reduce unnecessary work before reducing the game's identity: hidden geometry, overly broad rebuilds, material variation that causes extra surfaces, excessive transparent foliage, distant animation, and too-frequent activity decisions. Then adjust shadows, internal resolution, vegetation density, and far-view detail.

Keep clear terrain silhouettes, attractive building proportions, reliable edits, and controller response. Do not solve a performance problem by removing caves or discarding manual building details without an explicit design change.

12. Saves, undo, and resilience

Save model

Use local slots with a small metadata file and versioned world data: seed, generator version, modified chunks, building recipes, overrides, place definitions, water/path curves, presentation settings, and essential inhabitant identity. Runtime meshes and current pathfinding jobs are disposable caches.

Persist enough generator identity or baked source data to reproduce older saves. Do not assume the same seed recreates the same world after an algorithm or asset-library change. Unknown material/style IDs need an explicit fallback without destroying the original reference.

Atomic checkpoints

Stage a new checkpoint, write its data, verify completion, and then publish a manifest that points to the complete generation. Keep the previous complete checkpoint until the new one is recoverable. A simple rename of one metadata file is not sufficient if it references terrain data still being written in a separate store.

Serialize edits and checkpoint capture at a consistent revision, or snapshot the affected records. Never save half of a command that changes terrain, a bridge, and a water boundary together.

Proposed autosave policy: after a short idle period following committed edits, plus a periodic checkpoint and an attempt on suspension. Show unobtrusive saving state. Do not depend on a clean Quit action. Test interrupted saves, low storage, invalid data, and recovery from the last complete checkpoint.

Undo and redo

One brush stroke, structural drag, decoration edit, style change, or combined river operation equals one command. Cancellation restores the pre-preview state. Undo/redo must include dependent data, not just the visible mesh.

Begin with a bounded history, for example 50 commands within a 128 MiB budget, preserving at least the most recent allowed operation. Estimate the cost of unusually large edits before accepting them. Compress terrain deltas or store affected-chunk snapshots, and do not allow unbounded history to consume the device.

Session history can initially reset after loading; committed creative edits never do. Clearly separate user-facing undo from crash recovery. Keep backups of test worlds outside the active save directory during migration work.

13. Agent-assisted development and MCP

Is the MCP route practical?

Yes, through a **community integration**, rather than assuming MCP is a native Godot editor feature. Godot AI exposes live editor operations to clients including Hermes. Its tagged v3.2.5 instructions specify Godot 4.5+ with 4.7+ recommended, a Python server supplied through uv, and client configuration from the plugin dock. [S8]

Version note: the main README describes a v4 line, while the release listing inspected for this draft labels v3.2.5 as latest. Follow the requirements shipped with the chosen tag rather than applying the main branch's Godot 4.7+ requirement to every available release. [S8, S18, S19]

Recommended topology

For the first milestone, run the Godot editor, its MCP bridge, the project checkout, and the coding-agent process on the same development machine. The model itself may still be remote. This removes cross-machine path and loopback assumptions while establishing a working feedback loop.

Hermes can later orchestrate a dedicated development worker from the existing server. Hermes documents both subprocess and remote MCP configurations, but putting Hermes on Unraid does not make a desktop editor's localhost service automatically reachable. Use an authenticated tunnel or a deliberately configured private connection only after the local setup works. [S20]

MCP is not the Android deployment or measurement pipeline. Keep terminal access, Git, engine command-line runs, and a separate device-install/log workflow. Godot documents headless imports and command-line exports; use those for reproducible checks. [S21]

Installation and smoke test

Install a tagged Godot AI package and its matching requirements. Confirm the plugin directory layout, enable it in Godot, and use the dock-generated configuration for the selected client. Review the configuration diff and set project scope where supported. Keep the bridge private, disable unwanted telemetry, and do not include it in the shipping game's runtime dependencies. [S8, S19]

On a disposable scene, ask the agent to inspect the scene tree, add and save a camera, edit a script, report an intentional error, fix it, and run the scene. Verify changes on disk and after editor restart. Then test a screenshot or other live-view operation available in that exact release. Failed tests are blockers, not reasons to claim the agent “probably” has editor access.

Division of work

The agent implements bounded tickets, tests them, exports builds, and reports evidence. The player chooses visual references, judges controls on the actual device, reviews architecture changes, and decides whether the result feels pleasant. Automation can report a successful input action; it cannot establish that nudging a tiny window feels good from the sofa.

Use one active writer per subsystem and small branches. Each ticket ends with changed files, tests run, known limitations, screenshots where relevant, and an APK or deployment steps when the feature affects Android. No silent engine upgrades, new plugin families, save-format replacements, or world-size increases.

14. Milestones and acceptance gates

M0 — Platform and dependency proof

Build the smallest Android scene with the selected voxel backend, basic controller camera, a volumetric add/remove brush, one tunnel, sunlight, and a small water test surface. Save and reload a changed chunk. Verify desktop and Android debug/release library loading. Benchmark Mobile versus Compatibility. Separately complete the MCP smoke test.

Exit: a reproducible APK runs on the actual Thor, controller input works, terrain survives reload, required native libraries load, and measured frame/memory data is recorded. If the extension fails, test the supported module route or a compatible pinned version before choosing any engine change. Do not build a custom voxel engine as an unapproved workaround.

M1 — One excellent editable cottage

Implement one rectangular building, one roof profile, procedural details, stable surface IDs, manual overrides, suppression, and transactional resizing. Add the controller handles and a small terrain pad.

Exit: the moved/replaced/deleted-window test from section 5 passes after resize, undo/redo, save/load, and duplication. The player can comfortably perform it using only a controller.

M2 — A small place worth visiting

Combine a 128-metre riverside scene, cottage kit, paths, simple bridge, vegetation placement, a café function, four humans, two ducks, useful sound, and dependable saves. Include basic add/remove terrain editing and a visible cave/overhang demonstration carried forward from M0.

Exit: the representative session works from cold launch without touch input. The result is enjoyable enough to revisit, even before every landscape tool exists.

M3 — Full valley-editing freedom

Expand to the provisional 256-metre valley after benchmarking. Add the complete brush set, terrace/flatten/smooth, tunnel and cutaway tools, editable river/lake boundaries, a waterfall, foundation responses, and robust dirty-region navigation.

Exit: a tunnel under a lake, an overhang beside buildings, a moved riverbank, a deleted bridge, and a large undo all behave correctly. No hidden substitution of a heightmap-only system.

M4 — More life and architectural range

Add home, inn, playground, and park purposes; improve activity reservations and animation; deliver the second complete architectural kit. Expand detail editing and connected building forms based on actual use.

Exit: the same place function works in both styles, existing saves remain intact, and the normal benchmark population behaves without blocking edits. Art quantity grows only after the data-driven style boundary is proven.

M5 — A dependable wind-down game

Polish golden-hour lighting, optional time controls, camera bookmarks, UI scale, photo view, controller remapping, external-pad reconnect, TV behaviour, quiet mode, and save recovery. Run the full thermal and stress suite.

Exit: a 30-minute couch session has no need for touch, no lost edits, no persistent navigation failures, and no unexplained input or render stalls. Record which performance modes actually pass; leave unproven higher settings labelled experimental.

Planning rule

No calendar estimate until M0 and M1 reveal the two largest unknowns: device/backend reliability and non-destructive building complexity. Every milestone must leave a playable build, not only a framework for the next one.

15. Risks, decisions, and first handoff

Main risks

Risk	Response
Android voxel integration is less reliable than desktop.	Prove it first; pin a working engine/backend/export-template combination.
Resizing erases or misplaces personalised details.	Stable IDs, attachment policies, conflict recovery, and regression tests before content expansion.
Terrain edits cause stalls or stale collision/navigation.	Bounded dirty regions, revisioned jobs, coalesced updates, explicit pending state.
Beautiful lighting only works with desktop-only features.	Mobile visual target from day one; compare device captures, not only PC screenshots.
Controller selection feels tedious.	Test handles, snap strength, overlap cycling, and precision mode in M1, not at the end.
Water or inhabitants expand into major simulation projects.	Authored water rules and small activity-state machines; no economy or flood simulation.
Agent scope drifts into a large generic engine.	One milestone at a time; evidence-based tickets and approval for dependency or architecture changes.

Decisions to revisit after prototypes

Final terrain cell size and mesher; reliable engine/extension package; achievable 60 fps settings; preferred camera distance and snap increments; whether the second style should lean alpine or northern; and whether enterable interiors or literal shell-voxel editing are worth later effort. None prevents starting M0.

First implementation instruction

“Read this design and implement M0 only. Start by verifying the engine and native voxel dependency on desktop and Android ARM64. Create a minimal controller-driven test scene, terrain add/remove with a tunnel, chunk save/load, and a performance overlay. Keep MCP setup a separate development smoke test. Record exact versions, build steps, actual device results, and unresolved failures. Do not implement villagers, a catalogue of buildings, complex water, or a custom voxel engine yet.”

Definition of done for every ticket

A small reviewable change; acceptance tests that were actually run; no unreported errors; clean save/undo behaviour where applicable; a controller route for the feature; reproducible build instructions; and a short evidence report separating desktop checks from Thor checks. A passing parser is not proof that gameplay, rendering, and recovery work.

Creative north star: make one lovely riverside hamlet, then give it more freedom. Keep the valley; lose the bureaucracy.

Sources and research notes

Research checked on **6 September 2026**. Version references describe the pages inspected on that date and must be rechecked before installation. No plugin integration, engine benchmark, or APK has been executed as part of this document.

[S1] **AYN — Thor current product listing**. Current Max memory/storage options, advertised OS and battery.
Open source: www.ayntec.com

[S2] **AYN — original Thor specification graphic, reproduced by Retro Dodo**. Directly inspected manufacturer table: processor, clocks, memory, screens, cooling, connectivity, dimensions, output, and original storage claim.
Open source: storage.ghost.io

[S3] **Godot — official release archive**. Stable 4.7.2 and development 4.8 status.
Open source: godotengine.org

[S4] **Godot — overview and feature comparison of renderers**. Mobile rendering feature boundaries.
Open source: docs.godotengine.org

[S5] **Zylann — Voxel Tools repository**. Volumetric editing and chunk-mesh architecture.
Open source: github.com

[S6] **Voxel Tools — blocky terrains**. Mesher behaviour, fluid-model limitations, and blocky terrain details.
Open source: voxel-tools.readthedocs.io

[S7] **Godot — exporting for Android**. Export setup and platform workflow.
Open source: docs.godotengine.org

[S8] **Godot AI — tagged v3.2.5 README**. Release-specific requirements, client configuration, Hermes support, and network/telemetry notes.
Open source: github.com

[S9] **Voxel Tools — getting the module / extension**. GDExtension platform list, export templates, and maturity caveat.
Open source: voxel-tools.readthedocs.io

[S10] **Voxel Tools — v1.7 release**. Godot 4.7.2 module build; not a substitute for inspecting extension binaries.
Open source: github.com

[S11] **GUT — maintainer repository**. GDScript test framework and engine-version compatibility table.
Open source: github.com

[S12] **Godot — using navigation meshes**. Runtime baking and navigation geometry considerations.
Open source: docs.godotengine.org

[S13] **Input Helper — maintainer repository**. Input-device detection and rebinding helpers.
Open source: github.com

[S14] **Godot — using MultiMeshInstance3D**. Repeated mesh rendering and visibility considerations.
Open source: docs.godotengine.org

[S15] **Unity — Android documentation**. Alternative engine's Android platform support.
Open source: docs.unity3d.com

[S16] **Terrain3D — maintainer repository**. Heightmap-based terrain approach and supported editing tools.
Open source: github.com

[S17] **Voxel Tools — smooth terrains**. Separate smooth-terrain and LOD approach.
Open source: voxel-tools.readthedocs.io

[S18] **Godot AI — release listing**. v3.2.5 labelled latest in the inspected release listing.
Open source: github.com

[S19] **Godot AI — main README**. Describes a v4 line with different requirements; do not mix with a tagged v3 installation.
Open source: github.com

[S20] **Nous Research — Hermes MCP documentation**. Local and remote MCP configuration.
Open source: github.com

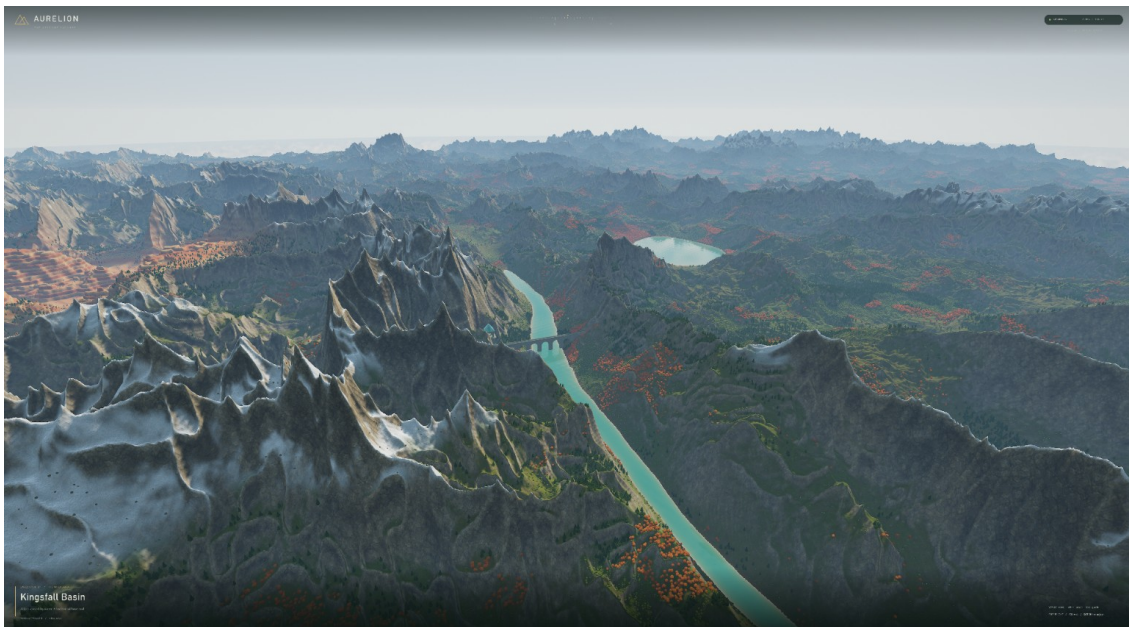
[S21] **Godot — command-line tutorial**. Headless imports, scripts, and exports.
Open source: docs.godotengine.org

Visual reference appendix

Supplied inspiration images. These establish landform, colour, and atmosphere only; they are not game screenshots or production assets. The bridge reference appears on the cover.



Terraces and material contrast. Warm stepped cliffs, scattered vegetation, and deep atmospheric separation. Borrow the rhythm and colour relationships, not the visible world size.



A valley with convincing depth. A turquoise river connects layered mountain forms. Recreate this sense of place within a finite editable valley and a lightweight scenic backdrop.

Visible reference labels: AURELION; Kingsfall Basin; The Old Crossing. The first image includes Reddit attribution to u/LostRequirement4828 in r/codex. Retain these images as private project references; do not extract or repackage their assets.